L1.5b — Linux pe serverul tău EC2

Docker, deploy pe AWS și nginx · exersezi comenzile Linux înainte de pasul 6

Lucrezi pe serverul `devops-initiere` de la pasul 5: intri prin `ssh` din terminalul VS Code (zsh) și muți fișiere prin **FileZilla** (SFTP) de pe Mac. Scrii fiecare comandă de mână, nu o copiezi, și te uiți la ce afișează. Peste tot, *IP-UL-SERVERULUI* = adresa publică de acum a instanței.

0. Pornești serverul și intri

În consola EC2: bifezi `devops-initiere` → **Instance state** → **Start instance**. Aștepți `Running` și copiezi noul **Public IPv4 address**.

```
ssh -i ~/.ssh/devops-initiere.pem ubuntu@IP-UL-SERVERULUI
```

Trebuie să vezi: promptul `ubuntu@ip-172-31-…:~$`. `Connection timed out` = IP-ul tău de acasă s-a schimbat: în **Security groups** → regula `22` → **Edit inbound rules** → **Source** = `My IP` → **Save rules**.

1. Unde ești și cine ești

```
whoami
hostname
pwd
uname -m
grep PRETTY /etc/os-release
uptime
date
```

Trebuie să vezi: `ubuntu`, `ip-172-31-…`, `/home/ubuntu`, `x86_64`, `PRETTY_NAME="Ubuntu 24.04… LTS"`, de cât timp merge serverul și data în `UTC`.

```
cat ~/.ssh/authorized_keys
```

Trebuie să vezi: un singur rând `ssh-rsa AAAA… devops-initiere`: jumătatea publică a cheii tale `.pem`. De asta te lasă să intri fără parolă.

2. Te miști prin foldere

```
ls /
cd /etc
pwd
ls | head
cd ..
pwd
cd ~
pwd
cd -
cd
ls -la
```

Trebuie să vezi: în `/` folderele `bin`, `etc`, `home`, `tmp`, `var`…; `cd ..` te duce în `/`; `cd ~` și `cd` singur te duc acasă (`/home/ubuntu`); `cd -` te întoarce unde erai și afișează folderul. `ls -la` arată și fișierele ascunse, cu punct în față: `.bashrc`, `.ssh`, plus folderul `devops-initiere`.

Scrii `cd /e` și apeși **Tab**: se completează singur `/etc/`. Apeși **Tab** de două ori pe `ls /u`: vezi variantele. Folosești **Tab** de aici încolo.

3. Instalezi un program cu apt

```
sudo apt update
sudo apt install -y tree
tree --version
tree -L 1 /
```

Trebuie să vezi: `tree v2.…`, apoi folderele din `/` desenate ca arbore. `sudo` = rulezi comanda ca `root`; fără el, `apt install` dă `Permission denied`: încearcă și tu `apt install tree` fără `sudo`.

4. Foldere și fișiere

```
mkdir exercitii
cd exercitii
mkdir -p proiect/src/main
touch proiect/README.md
echo "prima linie" > note.txt
echo "a doua linie" >> note.txt
cat note.txt
tree
```

Trebuie să vezi: cele două linii din `note.txt` și arborele cu `note.txt`, `proiect/README.md` și `proiect/src/main`. `>` scrie peste fișier, `>>` adaugă la final; `mkdir -p` face toate folderele din cale deodată.

```
cp note.txt copie.txt
mv copie.txt proiect/
mv proiect/copie.txt proiect/notite.txt
cp -r proiect proiect-backup
rm proiect/notite.txt
rm -r proiect-backup
tree
```

Trebuie să vezi: `notite.txt` și `proiect-backup` au dispărut. `mv` mută și redenumește; `-r` = cu tot ce e înăuntru. Pe Linux nu există Coș de gunoi: ce ștergi cu `rm` nu se mai întoarce.

5. Citești fișiere

```
cat -n /etc/hostname
head -n 5 /etc/passwd
tail -n 3 /etc/passwd
wc -l /etc/passwd
less /etc/services
```

Trebuie să vezi: primele 5 și ultimele 3 rânduri din `/etc/passwd` (un utilizator pe rând) și numărul de rânduri. În `less`: săgeți și **Space** ca să derulezi, scrii `/http` + **Enter** ca să cauți, `n` = următorul rezultat, `q` = ieși.

6. Editezi cu nano

```
nano ~/exercitii/proiect/README.md
```

Scrii trei rânduri: numele tău, data de azi și `Server: devops-initiere`. **Ctrl+O** → **Enter** = salvezi; **Ctrl+X** = ieși. Apoi un fișier al sistemului, cu `sudo`: mesajul afișat la fiecare intrare pe server.

```
sudo nano /etc/motd
```

Scrii `Serverul de DevOps al lui NUMELE-TAU - nu uita Stop!`, salvezi, ieși, apoi `exit` și intri din nou cu `ssh`.

Trebuie să vezi: mesajul tău în textul de bun venit, chiar înainte de prompt.

7. Editezi cu vim

Pe servere găsești aproape mereu `vim`, chiar și unde nu e `nano`. Are moduri: pornește în modul **Normal** (tastele sunt comenzi), `i` te trece în **Insert** (scrii text), **Esc** te întoarce în Normal, `:` deschide linia de comandă jos.

```
vim ~/exercitii/vim.txt
```

Apeși `i`, scrii pe rânduri separate `unu`, `doi`, `trei`, `patru`, `cinci`, apoi **Esc** și `:wq` + **Enter** (salvezi și ieși).

Trebuie să vezi: jos `-- INSERT --` cât scrii; după **Esc** dispare; după `:wq`, `"vim.txt" … written` și ești înapoi în terminal. `cat ~/exercitii/vim.txt` arată cele 5 rânduri.

Deschizi din nou fișierul și faci, în modul Normal, pe rând:

```
:set number         numerele rândurilor în stânga
j  k  h  l          jos / sus / stânga / dreapta (sau săgețile)
G   gg              la ultimul rând / la primul rând
3G                  la rândul 3
0   $               la începutul / la sfârșitul rândului
dd                  șterge rândul curent
u   Ctrl+R          anulează / refă
yy  p               copiază rândul / îl lipește dedesubt
o                   rând nou dedesubt, direct în Insert
x                   șterge caracterul de sub cursor
/trei  n            caută „trei” / următorul rezultat
:%s/doi/DOI/g       înlocuiește peste tot doi cu DOI
:w   :q   :q!       salvezi / ieși / ieși fără să salvezi
```

Trebuie să vezi: fiecare comandă schimbă imediat textul sau mută cursorul; după `:%s` jos apare câte înlocuiri a făcut. Dacă ai stricat ceva, `:q!` și fișierul rămâne cum l-ai salvat ultima dată. Ai scris litere în text fără să vrei = erai în Insert: **Esc**, apoi `u`. `E37: No write since last change` = încerci `:q` cu modificări nesalvate.

Exersezi pe fișierul tău: șterge rândul `patru`, copiază rândul `unu` la final, adaugă după `trei` rândul `trei si jumatate`, înlocuiește `cinci` cu `5`, salvează și ieși; `cat` arată rezultatul. Pentru mai mult: `vimtutor` (lecția oficială, ~30 de minute, ieși cu `:q`).

8. Pipe și redirecționări

```
cut -d: -f1 /etc/passwd | sort | head -n 5
cut -d: -f7 /etc/passwd | sort | uniq -c
ls /etc | wc -l
ls /nu-exista
ls /nu-exista 2> erori.txt
cat erori.txt
ls /nu-exista 2> /dev/null
```

Trebuie să vezi: primii 5 utilizatori în ordine alfabetică; câte conturi au fiecare shell (`/bin/bash`, `/usr/sbin/nologin`…); câte intrări sunt în `/etc`. `|` dă ieșirea unei comenzi ca intrare celei următoare; `2>` trimite erorile în fișier; `/dev/null` = le aruncă, ultima comandă nu afișează nimic.

9. Cauți cu grep și find

```
grep ubuntu /etc/passwd
grep -n Port /etc/ssh/sshd_config
grep -i image ~/devops-initiere/docker-compose.yaml
grep -r "8111" ~/devops-initiere
find ~ -name "*.txt"
find /etc -name "*.conf" | head
sudo find /var/log -type f -size +1M
```

Trebuie să vezi: `ubuntu:x:1000:1000:…:/home/ubuntu:/bin/bash`; `#Port 22` cu numărul rândului (`-n`); cele patru rânduri `image:` din compose (`-i` = fără să conteze literele mari); fișierul și rândul unde apare `8111` (`-r` = în tot folderul); `note.txt` și `erori.txt`; fișierele `.conf`; logurile mai mari de 1 MB (poate niciunul).

10. Permisiuni

```
cd ~/exercitii
ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
ls -l note.txt
```

Trebuie să vezi: întâi `-rw-------`, apoi `-rw-r--r--`, urmat de `ubuntu ubuntu` (proprietar și grup). Citești în grupuri de câte trei: proprietar / grup / ceilalți; `r` = 4, `w` = 2, `x` = 1, deci `6` = `rw-`, `4` = `r--`, `7` = `rwx`. Exact de asta ai dat `chmod 400` pe cheia `.pem`.

```
echo '#!/bin/bash' > salut.sh
echo 'echo "Salut de pe $(hostname), sunt $(whoami)"' >> salut.sh
./salut.sh
chmod +x salut.sh
ls -l salut.sh
./salut.sh
```

Trebuie să vezi: prima rulare dă `Permission denied`; după `chmod +x` apare un `x` în permisiuni și scriptul afișează `Salut de pe ip-172-31-…, sunt ubuntu`.

```
sudo chown root:root note.txt
ls -l note.txt
echo "test" >> note.txt
sudo chown ubuntu:ubuntu note.txt
```

Trebuie să vezi: `root root` și `Permission denied` la scriere: fișierul nu mai e al tău. După ultimul `chown` e iar al lui `ubuntu`.

11. Utilizatori

```
id
echo "pentru toti" > /tmp/public.txt
echo "doar pentru mine" > /tmp/secret.txt
chmod 644 /tmp/public.txt
chmod 600 /tmp/secret.txt
sudo useradd -m -s /bin/bash elev
sudo su - elev
whoami
cat /tmp/public.txt
cat /tmp/secret.txt
ls /home/ubuntu
exit
```

Trebuie să vezi: la `id` grupurile `sudo` și `docker` (de aceea merge `docker` fără `sudo`); ca `elev`: `whoami` = `elev`, `public.txt` se citește, `secret.txt` și `/home/ubuntu` dau `Permission denied`; după `exit` ești iar `ubuntu`.

```
sudo userdel -r elev
grep elev /etc/passwd
```

Trebuie să vezi: nimic la `grep`: contul e șters (un mesaj despre `mail spool` e normal).

12. Procese

```
sleep 600 &
sleep 700 &
jobs
ps aux | grep sleep
kill %1
jobs
pkill sleep
jobs
```

Trebuie să vezi: `[1]` și `[2]` cu câte un număr (PID-ul); `jobs` = două `Running`; `ps aux` le arată cu PID-urile lor (plus rândul cu `grep` însuși); după `kill %1` primul e `Terminated`, după `pkill sleep` nu mai e niciunul. `&` = pornește comanda în fundal și îți lasă promptul liber.

```
top
```

Trebuie să vezi: procesele ordonate după CPU și rândul `MiB Mem`; `q` = ieși. Pe o comandă care nu se mai termină, **Ctrl+C** o oprește: încearcă pe `sleep 100`.

13. Disc și memorie

```
df -h /
du -sh ~/exercitii
sudo du -sh /var/log
free -h
```

Trebuie să vezi: discul de `~19G` cu cât e folosit (`Use%`); cât ocupă folderul tău și logurile; la `free -h` rândul `Swap:` cu `2.0Gi` de la pasul 5. `-h` = cifre cu `K/M/G`.

14. Servicii și loguri

```
systemctl status docker
systemctl status ssh
sudo journalctl -u docker -n 10 --no-pager
who
```

Trebuie să vezi: la amândouă `Active: active (running)` și `enabled` (pornesc singure după **Start**); ieși din `status` cu `q`; ultimele 10 rânduri de log ale Docker; la `who`, tu: `ubuntu pts/0 … (IP-ul tău)`.

15. Rețea și aplicațiile tale

```
ip -br a
curl -s https://checkip.amazonaws.com
cd ~/devops-initiere
docker compose up -d
ss -tln
curl -s localhost:8111/api/users | head -c 200; echo
```

Trebuie să vezi: la `ens5` adresa privată `172.31.…`; `checkip` afișează adresa publică (aceeași ca în consolă); la `ss -tln` rânduri `LISTEN` pe `0.0.0.0:22`, `:8111`, `:8104`, `:8105`; `curl` întoarce JSON cu `"firstName":"Bogdan"`. Dacă primești `Connection refused`, Spring încă pornește: așteptă 30 de secunde și repetă.

```
docker compose down
```

16. Variabile, alias, istoric

```
echo $HOME
echo $USER
export ORAS=Bucuresti
echo "Lucrez din $ORAS"
history | tail -n 5
!!
```

Trebuie să vezi: `/home/ubuntu`, `ubuntu`, `Lucrez din Bucuresti`, ultimele 5 comenzi; `!!` = repetă ultima comandă. **Ctrl+R** + câteva litere caută în istoric; **Ctrl+L** curăță ecranul.

```
nano ~/.bashrc
```

Cu **Ctrl+W**, apoi **Ctrl+V**, sari la ultimul rând și adaugi pe ultimul rând: `alias dps='docker ps --format "{{.Names}}: {{.Status}}"'`. Salvezi și ieși.

```
source ~/.bashrc
dps
exit
```

Trebuie să vezi: după `source`, `dps` merge (nicio linie dacă nu ai containere pornite). Intri din nou cu `ssh`: `dps` merge încă, `echo $ORAS` e gol. `export` ține doar până ieși; ce e în `.bashrc` rămâne.

17. Arhive

```
cd ~
tar -czf exercitii-$(date +%F).tar.gz exercitii
ls -lh *.tar.gz
tar -tzf exercitii-*.tar.gz
mkdir restaurat
tar -xzf exercitii-*.tar.gz -C restaurat
diff -r exercitii restaurat/exercitii && echo IDENTICE
```

Trebuie să vezi: `exercitii-2026-…tar.gz` cu mărimea în `K`, lista fișierelor din arhivă, apoi `IDENTICE`. `c` = creează, `x` = extrage, `t` = listează, `z` = comprimat gzip, `f` = numele arhivei.

18. FileZilla: te conectezi prin SFTP

De pe `filezilla-project.org` → **Download FileZilla Client** → varianta pentru macOS → muți **FileZilla** în **Applications**. Prima dată îl deschizi cu click dreapta → **Open**.

File → **Site Manager** → **New site**, nume `devops-initiere`:
• **Protocol**: `SFTP – SSH File Transfer Protocol` • **Host**: *IP-UL-SERVERULUI* • **Port**: `22`
• **Logon Type**: `Key file` • **User**: `ubuntu` • **Key file**: **Browse** → `Downloads/devops-initiere.pem` → **Connect**.

Trebuie să vezi: fereastra `Unknown host key` → bifezi **Always trust this host** → **OK**; jos `Status: Directory listing of "/home/ubuntu" successful`; în dreapta folderele `devops-initiere`, `exercitii`, `restaurat` și arhiva. Dacă te întreabă dacă convertește cheia → **Yes**, o salvezi ca `devops-initiere.ppk` lângă `.pem`. La fiecare **Start** al instanței schimbi **Host** cu IP-ul nou.

19. FileZilla: muți fișiere

• **Urci:** în stânga (Mac-ul tău) găsești `notite-L1.txt` din repo-ul tău, îl tragi în dreapta, în `/home/ubuntu/exercitii`. În terminal: `cat ~/exercitii/notite-L1.txt`.
• **Descarci:** tragi arhiva `exercitii-…tar.gz` din dreapta în `Downloads`.
• **Permisiuni:** click dreapta pe `salut.sh` → **File permissions…**.
• **Schimbi pe server:** în terminal, `echo "scris din ssh" >> ~/exercitii/note.txt`; în FileZilla **F5**, apoi **View/Edit** pe `note.txt`.
• **Fără sudo:** încerci să tragi un fișier în `/etc`.

Trebuie să vezi: conținutul notițelor tale pe server; arhiva în `Downloads`; valoarea numerică `775` (sau `755`) și bifele de **Execute**; noul rând în `note.txt`; la `/etc` `permission denied` în **Status**: FileZilla lucrează ca `ubuntu`, fără `sudo`.

20. Același lucru cu scp, de pe Mac

Al doilea terminal în VS Code (**+**), pe Mac-ul tău:

```
mkdir -p ~/de-pe-server
scp -i ~/.ssh/devops-initiere.pem ubuntu@IP-UL-SERVERULUI:~/exercitii/salut.sh ~/de-pe-server/
scp -i ~/.ssh/devops-initiere.pem -r ubuntu@IP-UL-SERVERULUI:~/exercitii/proiect ~/de-pe-server/
ls -R ~/de-pe-server
```

Trebuie să vezi: `salut.sh 100%`, apoi `README.md 100%`, apoi fișierele copiate pe Mac-ul tău. La `scp` sursa e prima, destinația a doua; `ubuntu@IP:` în față = pe server.

21. Provocări

Doar cu comenzile de mai sus; rezultatul fiecăreia îl arăți în terminal.

1. Cu o singură comandă, în `~/provocari/app` apar folderele `src/main`, `src/test` și `docs`; le arăți cu `tree`.

2. Afișezi doar numărul de conturi din `/etc/passwd` care au shell-ul `/bin/bash`.
3. Afișezi cele mai mari 3 foldere din `/var`, ordonate după mărime, fără nicio eroare pe ecran.
4. Scriptul executabil `~/info.sh` afișează, câte una pe rând: utilizatorul, data, IP-ul public, memoria liberă și spațiul liber pe `/`. Ieșirea lui o salvezi în `~/info.txt` și o descarci cu FileZilla.
5. Pornești trei `sleep` în fundal, afișezi PID-urile lor și le oprești pe toate cu o singură comandă.
6. Găsești în `/etc` fișierele `.conf` care conțin cuvântul `timeout`, fără literele mari să conteze și fără erori pe ecran.
7. Urci de pe Mac cu FileZilla un folder cu trei fișiere; pe server îl arhivezi, ștergi folderul, îl refaci din arhivă și dovedești că e la fel.
8. Faci fișierul `/tmp/doar-grupul.txt` pe care `ubuntu` îl citește și scrie, grupul lui doar îl citește, iar ceilalți nu au acces deloc; arăți permisiunile.
9. Afișezi ultimele 5 comenzi din istoric care conțin `docker`.
10. Pornești din nou aplicațiile cu compose și, cu o singură linie, numeri câți utilizatori întoarce `/api/users` (câte apariții are `firstName`).
11. Deschizi `~/.bashrc` cu `vim`, sari la ultimul rând, îl dublezi și transformi copia în `alias dimg='docker images'`, doar cu comenzile din secțiunea 7; salvezi, dai `source` și rulezi `dimg`.

22. Oprești serverul

```
cd ~/devops-initiere
docker compose down
exit
```

În FileZilla: **Server** → **Disconnect**. În consola EC2: `devops-initiere` → **Instance state** → **Stop instance** → **Stop**.
Trebuie să vezi: `Stopped`. Nu **Terminate**: la pasul 6 lucrezi tot pe el.

Gata când…

Ai trecut prin toate secțiunile, ai editat în `nano` și în `vim`, ai rezolvat cele 11 provocări, ai mutat fișiere în ambele sensuri cu FileZilla și cu `scp`, iar instanța e `Stopped`. Continui cu `L1-6-NGINX.pdf`.
În `notite-L1.txt` adaugi un rând:

```
L1.5b: chmod 640 înseamnă: ÎNLOCUIEȘTE | diferența dintre > și >>: ÎNLOCUIEȘTE | comanda pentru
provocarea 3: ÎNLOCUIEȘTE
```